

Étude des Méthodes de Gradient Adaptatif en NLP

Comparaison de 5 optimiseurs pour le fine-tuning de DistilBERT sur IMDB

✂ Technologies Clés :

PyTorch

HuggingFace

DistilBERT

Colab

Scikit-learn

👤 Réalisé par :

Mouad BOULATAR

Mohamed Amine GHAZLI

Ayoub HAMROUDI

Jad MOUSLIM

👤 Supervisé par :

Pr. Faouzia BENABBOU

📅 Date :

Juin 2026

Soutenance orale

Plan de la Présentation

01 Contexte & Problématique

02 Optimiseurs Étudiés

03 Méthodologie & Architecture

04 Résultats Principaux

05 Analyses de Sensibilité

06 Discussion Critique

07 Conclusion & Recommandations

Contexte & Problématique

Le Défi de l'Optimisation pour les Transformers

Au programme :

- ➔ Domination des Transformers en NLP
- ➔ Sensibilité au choix de l'optimiseur
- ➔ Objectifs scientifiques du projet



Constat & Problématique

Les architectures **Transformer** (BERT, DistilBERT) dominent le NLP moderne, mais leur fine-tuning reste sensible au choix de l'optimiseur.

Les gradients sont **bruités**, l'espace des paramètres est de **très haute dimension**, et la performance dépend fortement du *learning rate*.

Objectifs du Projet :



Vitesse de convergence



Stabilité de l'apprentissage



Consommation mémoire GPU



Capacité de généralisation

Les Optimiseurs Étudiés

5 Algorithmes, 5 Philosophies

Au programme :

- ➔ SGD, AdamW, AdaFactor
- ➔ LAMB & Lion
- ➔ Mécanismes clés de chacun

Les 5 Optimiseurs Étudiés

Optimiseur	Type	Année
SGD + Momentum	Non-adaptatif	Classique
AdamW	Weight decay découplé	2017
AdaFactor	Mémoire réduite	2018
LAMB	Trust ratio par couche	2019
Lion	Sign momentum	2023



Mécanisme clé

AdaFactor factorise la matrice des second-moments : mémoire **sous-linéaire**, idéal pour GPU limités.

Lion n'utilise que le **signe** du momentum : très léger en mémoire mais sensible au LR.



En résumé

AdamW reste la référence stable. **AdaFactor** économise la mémoire. **LAMB** cible les très grands batches. **Lion** mise sur la simplicité du signe. **SGD** sert de baseline non-adaptative.

Méthodologie & Architecture

Protocole Expérimental Standardisé

Au programme :

- ➔ Modèle DistilBERT & dataset IMDB
- ➔ Protocole expérimental fixe
- ➔ Pipeline de benchmark

Modèle, Dataset & Protocole Expérimental



Modèle & Dataset

DistilBERT : 66M paramètres, 6 couches Transformer.

IMDB : 8 000 train / 2 000 val. / 2 000 test, classes équilibrées 50/50.

Protocole fixe :



Seed = 42, 3 epochs



Batch = 16, LR = $2e-5$



Weight decay = 0.01, warmup = 10%



Mixed precision (AMP)



Figure : Architecture globale du pipeline NLP

Pipeline Expérimental & Suivi des Gradients



Benchmark Multi-Optimiseurs

Chaque optimiseur est évalué sous le **même protocole** :
mêmes données, même seed, mêmes hyperparamètres fixes.



Suivi des gradients

Norme L2 des gradients mesurée à chaque couche :
Embeddings, Layer 0, Layer 2, Layer 5, Classifier.

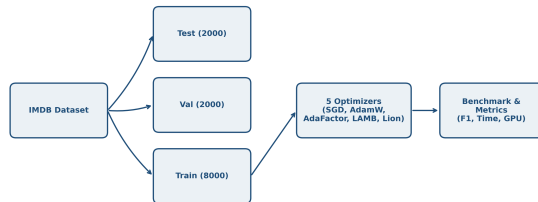


Figure : Pipeline expérimental du benchmark

Résultats Principaux

Benchmark Comparatif des 5 Optimiseurs

Au programme :

- ➔ Tableau comparatif final
- ➔ Courbes de convergence
- ➔ Classement des optimiseurs

Résultats Principaux — Tableau Comparatif

Optim.	F1	GPU	Temps
AdaFactor	89.72%	1204 MB	263.5s
AdamW	89.53%	1724 MB	250.6s
Lion	85.99%	1468 MB	243.8s
LAMB	81.52%	1719 MB	273.3s
SGD+Mom.	53.79%	1458 MB	235.6s



Classement Final

1. AdaFactor
2. AdamW
3. Lion
4. LAMB
5. SGD (échec convergence)

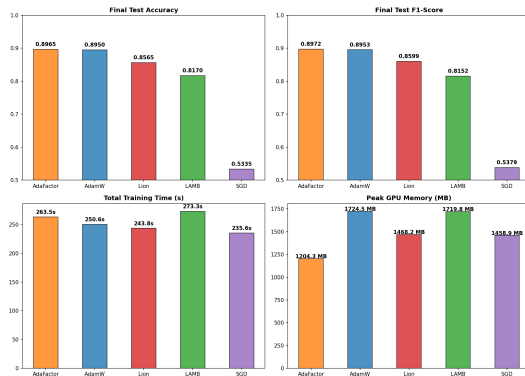


Figure : Comparaison finale des métriques

Courbes de Convergence



Observations Clés

AdamW & AdaFactor : convergence rapide et stable dès l'époch 1.

SGD : stagnation, forte variance, échec.

Lion : léger surapprentissage à l'époch 3.

LAMB : convergence lente.

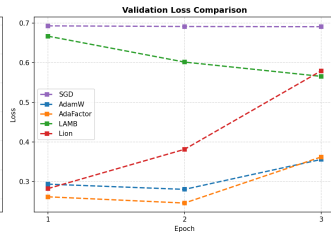
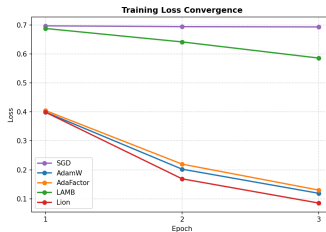


Figure : Courbes loss train/validation par epoch

Analyses de Sensibilité

Robustesse au LR et au Batch Size

Au programme :

- ➔ Sensibilité au learning rate
- ➔ Robustesse au batch size
- ➔ Stabilité comparée

Sensibilité au Learning Rate



Résultats Clés

AdamW & AdaFactor : stables sur toute la plage de LR testée ($1e-5$ à $1e-4$).

Lion : sensible mais performant à $LR=2e-5$ ($F1=0.9052$).

SGD : instable, quel que soit le LR.

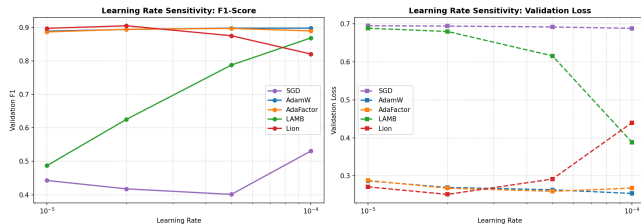


Figure : Sensibilité F1-Score et Loss selon le LR

Robustesse au Batch Size



Résultats Clés

AdaFactor : meilleure performance avec la **mémoire la plus faible**, à toutes les tailles (8, 16, 32).

LAMB & SGD : chute fortement sur petits batchs.

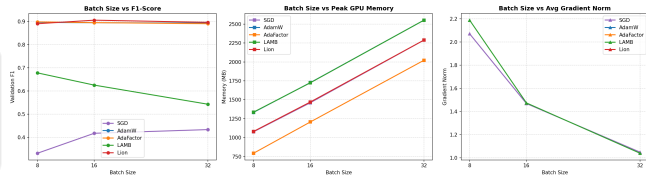


Figure : Robustesse F1 / mémoire selon le batch size

Conclusion & Recommendations



Réalisations Clés

- ✓ 5 optimiseurs comparés sous protocole strict
- ✓ Analyses de sensibilité LR & batch size
- ✓ Suivi des gradients par couche
- ✓ Pipeline 100% reproductible (seed=42)



Recommandation Finale

AdaFactor : meilleur compromis pour GPU limité (F1=89.72%, -30% mémoire vs AdamW).

AdamW : choix sûr et stable par défaut.

Perspective : valider sur MiniLM.



github.com/BOULATARM/projet16-adaptive-gradient-methods-nlp

FIN DE LA PRÉSENTATION

Merci pour votre Attention!

Place à présent à vos questions et remarques

 github.com/BOULATARM/projet16-adaptive-gradient-methods-nlp

 Groupe 16 :

**Mouad BOULATAR, Mohamed Amine GHAZLI,
Ayoub HAMROUDI, Jad MOUSLIM**

 Supervisé par :

Pr. Faouzia BENABBOU